

## Final Report

### 1. DESCRIPTION:

A cross-platform mobile app that recommends posts to users based on their progress in a tv series to avoid spoilers and promote a safe fandom experience. It will have a simple UI similar to Tumblr, where users can read and write text-based posts about fandoms they enjoy. We will implement Retrieval-Augmented Generation (Section 7) to flag spoilers and improve the user experience.

### 2. LIBRARIES

- **Frontend:** Flutter, Dart
- **Backend:** Express, TypeScript, Node.js
- **LLMs:**
  - **Platform:** Gemini API
- **Database:** TypeORM, PostgreSQL, hosted by Neon
- **Containerization:** Docker

### 3. FEATURES

- 1) User accounts and authentication
- 2) Text-based UI similar to Tumblr

#### Dimensions.Guide | Tumblr Post – Text



FIGURE 1. Sample text based UI.

- 3) Ability to input your current episode for a given tv show and subscribe / unsubscribe from shows
- 4) Recommendation of posts based on current episode in a series - spoiler control, tagging system

### 4. CHALLENGES

The biggest challenge was definitely the sheer scale of the project. We knew going into it that the idea was rather ambitious, but it didn't really click just how much it would be until we started working. It took a lot of time to get each of the features up and running, especially features related to the llm, so timelines were shifted and features were let go.

We also decided that one of us would do the backend and the other would do the frontend, so there was the issue of communicating what each of us needed from the other and finding the motivation to actually do things as requested.

## 5. LIMITATIONS

Going into it, we set out to have more user interaction and customization. We were going to have the ability to follow other users, liking and commenting on posts, and potentially profile customization. However, given the length of time it took to complete other tasks, we did not include any of these features. In future, we would like to implement them to make the app more complete.

There are also some rather inefficient systems in place. A lot of it would not scale outside of testing. There is little support for a large user base. We need to implement more methods of caching and data persistence.

## 6. CONCLUSION

While we did not accomplish everything we set out to do, this project was an incredible learning experience. We gained experience with vector-based databases, integrating spoiler detection with llms, github and pull requests, and communicating from different programming perspectives, among other things. There is much yet to do, but we are proud of the current state of the project.

## 7. RETRIEVAL AUGMENTED GENERATION

Retrieval-Augmented Generation is a method of allowing llms to respond to queries using supporting information beyond their training data.

Our implementation involves retrieving relevant transcript data from a database to inform llm spoiler detection. The data is sourced from online transcript websites which are then chunked and stored as vectors in our PostGreSQL database hosted on neon. Posts are tagged with a show, and when a user's feed is being generated, each show is checked for spoilers. We pass the post in question to a chunking model, then we query the database for context from transcripts for that show using cosine similarity. Then, the resulting context is used by the LLM to determine if the post is a spoiler for the current user.

Our database is modeled, generally, as such. We have users table, a posts table, and relevant join tables for likes and followers. Our vector database has a shows table, an episodes table, and a chunks table so that we can store indexed chunks for every episode of every show.

## REFERENCES

OKLAHOMA CITY UNIVERSITY, PETREE COLLEGE OF ARTS & SCIENCES, COMPUTER SCIENCE